

Функции, модули, Promise и async/await

Фундаментальное инженерное исследование лексического окружения, замыканий, модульной системы ESM, архитектуры микротасок Event Loop и паттернов асинхронного взаимодействия в React.

 zsh — async pipeline preview

```
// Асинхронный пайплайн загрузки с отменой через
AbortController:
async function
fetchUserData(userId, signal) {
const response =
await
fetch(`/api/users/${userId}`, { signal }); if (!response.ok)
throw new
Error(`HTTP ${response.status}`); return await response.json(); }
```

Продвинутые функции, Лексическое окружение и Каррирование

1. Виды объявления функций и Execution Context

В JavaScript способ объявления функции определяет момент ее инициализации в памяти и привязку контекста вызова:

- **Function Declaration:** Инстанцируется на этапе создания контекста исполнения (Фаза Hoisting). Доступна для вызова до физического места объявления в коде.
- **Function Expression:** Анонимная или именованная функция, присвоенная переменной. Инициализируется только тогда, когда поток выполнения доходит до соответствующей строки.
- **Arrow Function:** Стрелочная функция не имеет собственного контекста `this`, прототипа (`prototype`) и не создает псевдомассива `arguments`, захватывая `this` лексически из окружающего контекста.

🧩 2. Lexical Environment, Scope Chain и замыкания

Каждый вызов функции в JavaScript создает внутренний служебный объект — **Лексическое окружение (Lexical Environment)**, состоящий из двух компонентов:

1. **Environment Record:** Запись идентификаторов, хранящая локальные переменные, аргументы и объявления функций.
2. **Ссылка на внешнее окружение (Outer Lexical Environment Reference):** Ссылка на лексическое окружение, в котором функция была физически создана в коде.

Поиск переменной всегда идет снизу вверх по **Scope Chain (Цепочке областей видимости)**. Если переменная не найдена в текущем окружении, движок обращается к внешнему окружению вплоть до глобального объекта (`globalThis` / `window`).

🔗 3. Каррирование (Currying) и частичное применение в обработчиках React

Каррирование — это трансформация функции от нескольких аргументов вида `f(a, b, c)` в цепочку функций от одного аргумента: `f(a)(b)(c)`. В React этот паттерн применяется для создания параметризованных обработчиков событий без стрелочных функций в разметке:

```
CurriedHandler.jsx

// Фабрика обработчиков инпутов с фиксацией имени поля через
// замыкание:
const
handleFieldChange = fieldName => event
=> { const value =
event.target.value; setFormData(prev =>
({ ...prev, [fieldName]: value })); };

// В JSX вызов handleFieldChange("email") создает чистый
// колбэк:

```

Модульная система ECMAScript (ESM) и оптимизация сборки

1. Архитектурное сравнение: ESM против CommonJS

В мире Node.js и фронтенда исторически сосуществуют два стандарта модулей. Понимание их различий критично при настройке сборщика Vite и серверного рендеринга:

Критерий	CommonJS (CJS)	ECMAScript Modules (ESM)
Синтаксис	<code>require()</code> / <code>module.exports</code>	<code>import</code> / <code>export</code>
Момент загрузки	Динамический (в рантайме, синхронно)	Статический (на этапе парсинга, асинхронно)
Характер связывания	Копирование значений (Copy by Value)	Живые ссылки на сущности (Live Bindings)

Характер связывания	Копирование значений (Copy by Value)	Живые ссылки на сущности (Live Bindings)
Tree-Shaking	Практически невозможен (динамический вызов)	Глубокая оптимизация мертвого кода Rollup/esbuild
Среда исполнения	Классический Node.js	Современные браузеры и Vite из коробки

2. Динамический импорт: `import()` и Code Splitting

Статический `import ... from '...'` обязан располагаться строго на верхнем уровне файла. Для отложенной загрузки тяжелых модулей по требованию применяется функциональная форма **динамического импорта** `import()`, возвращающая Promise:

// 1. Динамическая загрузка тяжелой библиотеки графиков по клику:

```
async function  
renderAnalytics() {  
  const { ChartEngine } =  
  await  
  import("./heavyChartEngine.js"); const chart =  
  new  
  ChartEngine(); chart.draw(); }
```

// 2. В React на базе динамического импорта построен механизм React.lazy:

```
const AdminDashboard = React.lazy(() => import("./pages/AdminDashboard"));
```

Модель асинхронности: Call Stack, Web APIs и Microtasks

Движок JavaScript (V8) является **однопоточным**: в каждый момент времени выполняется ровно одна инструкция. Асинхронность достигается за счет взаимодействия Call Stack с системным окружением браузера (Web APIs) и очередями задач Event Loop.

Архитектура очередей Event Loop в V8:

Очередь микротасок опустошается целиком перед каждой макротаской и перед отрисовкой кадра (Render UI).

MICROTASK QUEUE (ВЫСШИЙ ПРИОРИТЕТ)

VIP

- `Promise.then() / catch() / finally()`
- `queueMicrotask()`
- `MutationObserver`

MACROTASK QUEUE (TASK QUEUE)

СТАНДАРТ

- `setTimeout() / setInterval()`
- Сетевые события `fetch / XHR`
- Пользовательские события (Click, Scroll)

```
console.log("1. Синхронный код");

setTimeout(() => console.log("4. Макротаска (setTimeout)",
0); Promise.resolve().then(() => console.log("3. Микротаска (Promise)"));
console.log("2. Синхронный конец");

// Порядок вывода в консоль:
// 1. Синхронный код
// 2. Синхронный конец
// 3. Микротаска (Promise) ← Микротаски выполняются ПЕРЕД
    макротасками!
// 4. Макротаска (setTimeout)
```

Анатомия Promise и методы параллельного исполнения

🔗 1. Внутренние состояния Promise

Promise — это специальный объект, представляющий результат незавершенной асинхронной операции. Объект Promise находится строго в одном из трех взаимоисключающих состояний (State Machine):

- `pending` (ожидание) — начальное состояние, операция еще выполняется.
- `fulfilled` (выполнено успешно) — операция завершилась вызовом `resolve(value)`.
- `rejected` (отклонено с ошибкой) — операция завершилась вызовом `reject(error)`.

После перехода в состояние `fulfilled` или `rejected` промис становится **settled (окончательно зафиксированным)** и никогда больше не может изменить свое состояние или значение.

2. Методы параллельного исполнения промисов

Метод	Когда завершается успешно	Поведение при ошибке (Fail-Fast)
<code>Promise.all()</code>	Когда ВСЕ промисы выполнились успешно. Возвращает массив результатов.	Мгновенный отказ (Fail-Fast) при первой же ошибке любого из промисов.
<code>Promise.allSettled()</code>	Когда ВСЕ промисы завершились (независимо от успеха/ошибки).	Никогда не отклоняется. Возвращает массив объектов <code>{ status, value/reason }</code> .
<code>Promise.race()</code>	Когда ПЕРВЫЙ промис перешел в любое состояние (успех или ошибка).	Отклоняется, если самый быстрый промис завершился с ошибкой.
<code>Promise.any()</code>	Когда ПЕРВЫЙ промис выполнен успешно .	Отклоняется с <code>AggregateError</code> только если ВСЕ промисы завершились ошибкой.

```
// 1. Параллельная загрузка независимых ресурсов (ускорение в 2
    раза!):
const [user, posts] =
await Promise.all([ fetchUser(userId),
fetchUserPosts(userId) ]);

// 2. Устойчивая загрузка независимых виджетов дашборда:
const results =
await Promise.allSettled([ fetchWeather(),
fetchCurrency() ]); results.forEach(res => { if (res.status ===
"fulfilled")
renderWidget(res.value);
if (res.status ===
"rejected")
renderErrorPlaceholder(res.reason); });
```

Синтаксис `async / await` и надежная обработка ошибок

Конструкция `async/await` — это синтаксический сахар над промисами и генераторами. Ключевое слово `async` перед функцией гарантирует, что она **всегда неявно возвращает Promise**. Ключевое слово `await` приостанавливает выполнение функции до разрешения промиса, не блокируя при этом главный поток браузера.

```
async function
loadProductCatalog() {
  try {
    const res =
    await
    fetch("https://api.site.com/products");

    // ⚠ ВАЖНО: fetch НЕ выбрасывает ошибку при статусах 404 или
    // 500!
    // Проверка response.ok обязательна вручную:
    if (!res.ok) {
      throw new
      Error(`Сервер ответил ошибкой со статусом: ${res.status}`); }

    const data =
    await res.json(); return data; }
    catch (err) { console.error("Сетевой сбой или некорректный JSON:",
    err.message); throw err;
    // Пробрасываем ошибку выше в UI слой
  } finally { console.log("Очистка состояния индикатора загрузки (Loading = false)"); } }
```

Асинхронные паттерны в React и предотвращение Race Condition

⚠ 1. Проблема Race Condition и утечек памяти при демонтировании

Классическая ошибка: пользователь быстро переключает вкладки (ID 1 $\rightarrow$ ID 2). Запрос для ID 1 уходит раньше, но из-за сетевой задержки отвечает *позже* запроса ID 2. В результате интерфейс перезаписывается устаревшими данными первого запроса (Race Condition).

Для решения проблемы применяется стандартный веб-интерфейс `AbortController` в сочетании с функцией очистки (Cleanup) хука `useEffect`:

```
import { useState, useEffect }
from
"react";

function
UserProfile({ userId }) {
  const [user, setUser] =
  useState(null); const [status, setStatus] =
  useState("idle");
  // 'idle' | 'loading' | 'success' | 'error'
  const [error, setError] =
  useState(null);

  useEffect(() => {
    // Создаем контроллер отмены для текущего эффекта:
    const controller =
    new
    AbortController();
```

```
async function
fetchData() {
  setStatus("loading"); try {
    const res =
    await
    fetch(`/api/users/${userId}`, { signal: controller.signal });
    if (!res.ok)
    throw new
    Error("Не удалось загрузить профиль"); const data =
    await res.json();

    setUser(data);
    setStatus("success"); } catch (err) {
      // Игнорируем ошибку отмены AbortError, если компонент
      размонтирован:
      if (err.name !==
      "AbortError") {
        setError(err.message);
        setStatus("error"); } } }

  fetchData();
```

```
// Функция очистки: вызывается при смене userId или  
размонтировании компонента:
```

```
return () => controller.abort(); }, [userId]);
```

```
// Перезапуск строго при изменении userId
```

```
if (status ===
```

```
"loading")
```

```
return <div>Загрузка профиля...</div>;
```

```
if (status ===
```

```
"error")
```

```
return <div
```

```
className="error">Ошибка: {error}</div>;
```

```
if (!user)
```

```
return
```

```
null;
```

```
return <h2>{user.name}</h2>; }
```